Instituto Federal de Educação, Ciência e Tecnologia
do Rio Grande do Norte

Ian de Araújo, Jardson Alan

Classificação do impacto das redes sociais com Naive Bayes e árvore de decisão

Rio Grande do Norte
2026

Ian de Araújo, Jardson Alan

Classificação do impacto das redes sociais com Naive Bayes e árvore de decisão

Artigo apresentado à disciplina de Análise de Dados do Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte, com o registro da etapa de classificação do projeto.

Rio Grande do Norte
2026

Classificação do impacto das redes sociais com Naive Bayes e árvore de decisão

Ian de Araújo[1] e Jardson Alan[1]

RESUMO

Esta etapa classifica o impacto das redes sociais em 4.500 estudantes. A classe é `Overall_Impact`, com os rótulos `Beneficial`, `Neutral` e `Negative`. As entradas são sete medidas numéricas, as mesmas do agrupamento, e o índice de saúde mental fica de fora. O Naive Bayes gaussiano é avaliado por validação cruzada de cinco partes, com acurácia média de 0,9238. A árvore de decisão passa por `GridSearchCV` no recorte de treino. Os hiperparâmetros escolhidos são critério de Gini, profundidade máxima 7, mínimo de 10 amostras para dividir um nó e mínimo de 4 amostras na folha. Com esses parâmetros, a validação cruzada da árvore chega a 0,9480. Os dois resultados usam `cross_val_score`.

Palavras-chave: Classificação. Naive Bayes. Árvore de decisão. Validação cruzada. Redes sociais.

1 ETAPA DE CLASSIFICAÇÃO

O objetivo é prever a classe já existente em `Overall_Impact`. Há 3.681 registros `Beneficial`, 654 `Neutral` e 165 `Negative`, como na Figura 1. O arquivo é `data/raw/dataset.csv`, com 4.500 estudantes (YADAV, 2026).

As entradas vêm de `prepare_data`, em `src/clustering/grouping.py`: idade, horas diárias, horas extras de fim de semana, duração e qualidade do sono, estresse percebido e GPA. Os vazios de estresse e de GPA recebem a média truncada por idade. `Student_ID` e `Mental_Health_Index` não entram. A classe também não entra como variável.

Os dois classificadores usam o mesmo recorte, com 33% dos registros para teste e semente 125. A comparação exigida pela etapa, porém, é a validação cruzada de cinco partes, com acurácia, aplicada aos dois modelos por `cross_val_score`.

2 NAIVE BAYES

O modelo é o `GaussianNB`, adequado às colunas numéricas. Não há busca de hiperparâmetros. A Tabela 1 mostra os cinco resultados e a média de 0,9238, com desvio-padrão de 0,0090. No recorte de teste, a acurácia foi 0,9178.

A Figura 2 mostra as mesmas cinco partes. O eixo começa em 0,85 para a diferença entre os modelos aparecer. Em todas as partes a árvore fica acima do Naive Bayes.

[1] Estudantes da disciplina de Análise de Dados do Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte.

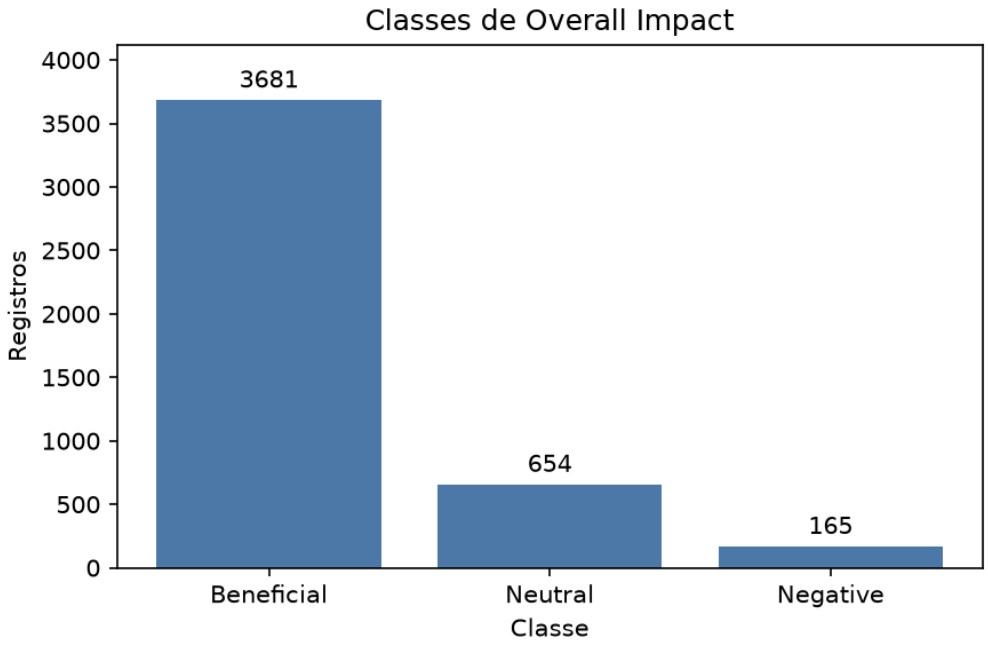


Figura 1 – Contagem das classes de `Overall_Impact`

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

Tabela 1 – Acurácia na validação cruzada de cinco partes

Modelo	Parte 1	Parte 2	Parte 3	Parte 4	Parte 5
Naive Bayes	0,9278	0,9289	0,9233	0,9067	0,9322
Árvore de decisão	0,9544	0,9444	0,9556	0,9356	0,9500

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

A Tabela 2 é a matriz do recorte de teste. Os 52 registros `Negative` desse recorte foram quase todos reconhecidos: 49 acertos. Dos 229 `Neutral`, 194 foram previstos nessa classe. A maior troca ocorre entre `Beneficial` e `Neutral`.

Tabela 2 – Matriz do Naive Bayes no recorte de teste

Real	Beneficial	Negative	Neutral
Beneficial	1120	0	84
Negative	0	49	3
Neutral	14	21	194

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

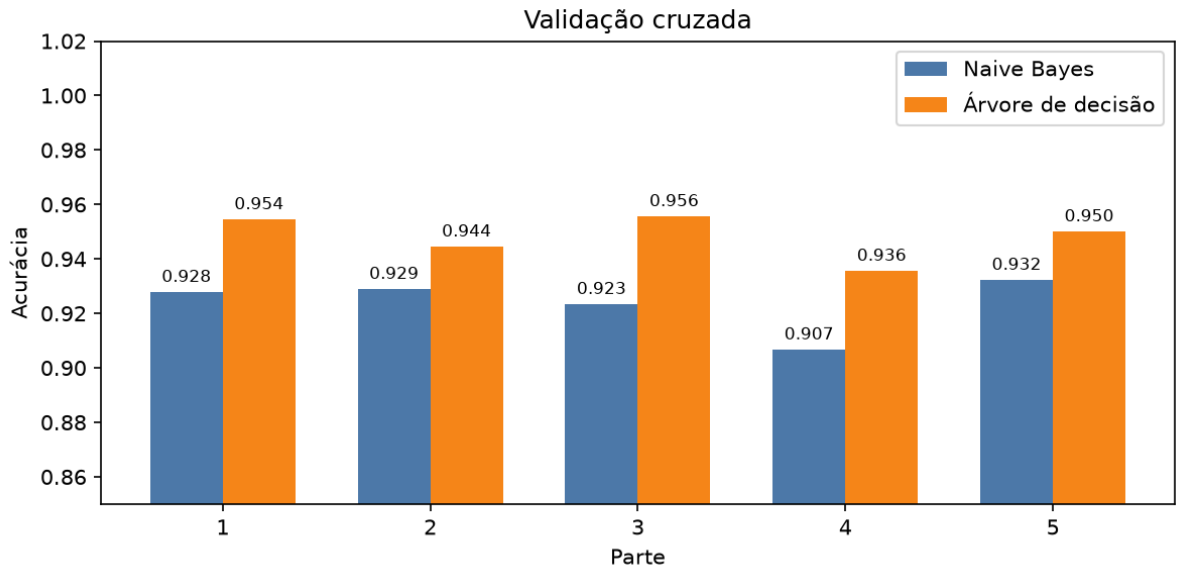


Figura 2 – Acurácia nas cinco partes da validação cruzada

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

3 ÁRVORE DE DECISÃO

O `GridSearchCV` ajusta a árvore só no recorte de treino. A grade combina critério `gini` ou `entropy`, profundidade máxima 3, 5, 7, 10 ou sem limite, mínimo de 2, 5 ou 10 amostras para dividir um nó e mínimo de 1, 2 ou 4 amostras na folha. São cinco partes internas e o critério da busca é a acurácia. A semente é 125.

O conjunto escolhido foi critério de Gini, profundidade máxima 7, mínimo de 10 amostras para dividir e mínimo de 4 amostras na folha. Uma árvore nova, com esses hiperparâmetros, foi avaliada por `cross_val_score` no conjunto inteiro. A média foi 0,9480 e o desvio-padrão 0,0074, como na Tabela 1. No recorte de teste, a acurácia foi 0,9434.

Tabela 3 – Matriz da árvore de decisão no recorte de teste

Real	Beneficial	Negative	Neutral
Beneficial	1186	0	18
Negative	0	44	8
Neutral	51	7	171

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

Na Tabela 3 e na Figura 3, a árvore acerta 1.186 dos 1.204 `Beneficial` do teste e 44 dos 52 `Negative`. O `Neutral` é a classe mais confundida: 51 desses registros foram previstos como `Beneficial`.

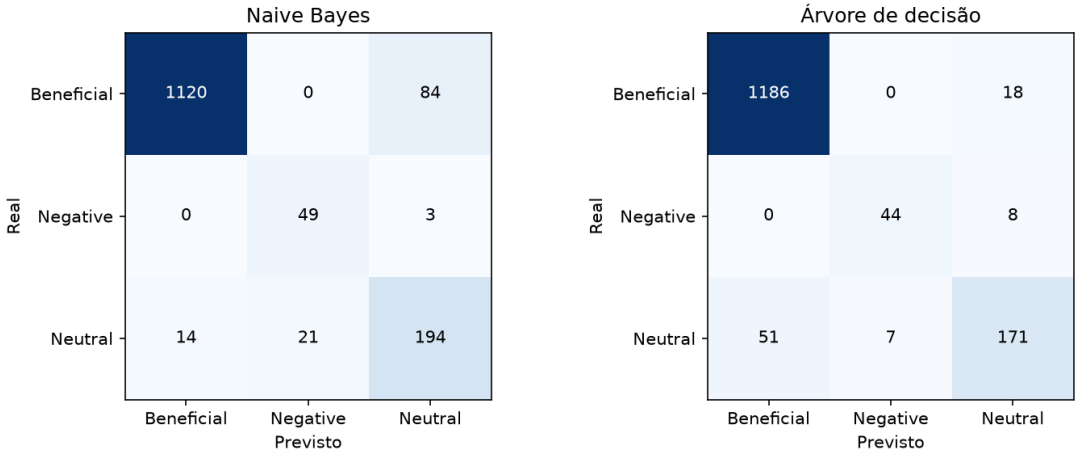


Figura 3 – Matrizes de confusão no recorte de teste

Fonte: Elaborado pelos autores (2026), pela execução dos classificadores.

4 COMPARATIVO

Pela validação cruzada, a árvore fica acima do Naive Bayes em todas as cinco partes. A média passa de 0,9238 para 0,9480. O desvio-padrão também é menor, 0,0074 contra 0,0090. A classe `Beneficial` é a maioria do arquivo, então a acurácia dos dois já parte de um patamar alto. A diferença aparece sobretudo nela: a árvore troca menos `Beneficial` com `Neutral`. O Naive Bayes, no recorte de teste, recupera um pouco mais os `Negative` (49 contra 44).

5 CÓDIGO UTILIZADO

As listagens são os arquivos desta etapa, como estão no repositório. `main.py` chama os dois classificadores e a função que grava as figuras. `naive_bayes.py` avalia o `GaussianNB` com `cross_val_score`. `tree_decision.py` escolhe os hiperparâmetros com `GridSearchCV` e avalia a árvore escolhida também com `cross_val_score`. `charts.py` grava os três gráficos em `reports/figures/`.

5.1 CHAMADA NO FLUXO

Arquivo `main.py`.

```
1 from src.data.load_data import load_data
2 from src.preprocessing.clean_data import select_numeric, null_values
3 from src.preprocessing.standardize_data import standardize_numeric_data
4 from src.visualization.charts import classification_charts,
      data_standardization_comparison
5 from src.clustering.grouping import run_grouping
6 from src.classifiers.naive_bayes import nb_classifier
```

```
7  from src.classifiers.tree_decision import tree_classifier
8
9  def main():
10   # Carrega os dados do arquivo CSV
11   data = load_data("data/raw/dataset.csv")
12   print("Dados carregados com sucesso")
13
14   # Seleciona os dados numéricos
15   data_numeric = select_numeric(data)
16   print("Dados numéricos selecionados com sucesso")
17
18   # Preenche os valores nulos com a média da coluna (filtrada pela idade)
19   data_null = null_values(data_numeric)
20   print("Valores nulos preenchidos com sucesso")
21
22   # Padroniza os dados numéricos
23   data_scaled = standardize_numeric_data(data_null)
24   print("Dados padronizados com sucesso")
25
26   # Compara os dados antes e depois da padronização
27   data_standardization_comparison(data, data_scaled)
28   print("Comparação dos dados antes e depois da padronização realizada com sucesso
       ")
29
30   # Executa o agrupamento e calcula as métricas
31   results, comparison, profile, kmeans_label_comparison,
       hierarchical_label_comparison = run_grouping(data)
32
33   # Executa o classificador Naive Bayes
34   nb_result = nb_classifier(data)
35   print("\nClassificador Naive Bayes executado com sucesso")
36
37   # Executa o classificador de árvore de decisão
38   tree_result = tree_classifier(data)
39   print("\nClassificador de árvore de decisão executado com sucesso")
40
41   # Gráficos da etapa de classificação
42   classification_charts(
43     data,
44     nb_result["scores"],
45     tree_result["scores"],
46     nb_result["comparison"],
47     tree_result["comparison"],
48   )
49   print("Gráficos da classificação gravados com sucesso")
50
51   print("\nResultados para escolha de k:")
```

```
    print(results)

    print("\nComparação entre algoritmos:")
    print(comparison)

    print("\nPerfil dos clusters:")
    print(profile)

    print("\nK-Means - Clusters versus Overall_Impact:")
    print(kmeans_label_comparison)

    print("\nHierárquico - Clusters versus Overall_Impact:")
    print(hierarchical_label_comparison)

if __name__ == "__main__":
    main()
```

5.2 NAIVE BAYES

Arquivo src/classifiers/naive_bayes.py.

```
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder

from src.clustering.grouping import prepare_data


def nb_classifier(data: pd.DataFrame):
    X, _, y = prepare_data(  # "_" indica que a segunda coluna nao sera utilizada
        data
    )  # Importacao de "grouping.py" (dados numericos a serem classificados)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.33, random_state=125
    )

    # Classificador GaussianNB
    model = GaussianNB()

    # Desempenho pela validação cruzada, no conjunto inteiro
    scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
    print("\nNaive Bayes")
    print("Validação cruzada (5 folds):", scores)
    print("Acurácia média:", scores.mean())
```

```
    print("Desvio-padrão:", scores.std())

    # Modelo de treinamento no recorte de treino
    model.fit(X_train, y_train)

    # Codificacao dos valores reais (nomes)
    encoder = LabelEncoder()
    encoder.fit(data["Overall_Impact"])

    predicted = model.predict(X_test)
    real_labels = encoder.inverse_transform(y_test)
    predicted_labels = encoder.inverse_transform(predicted)

    print("Acurácia no teste:", accuracy_score(y_test, predicted))

    print("\nRelatório por classe:")
    print(classification_report(
        y_test,
        predicted,
        target_names=encoder.classes_,
    ))

    comparison = pd.crosstab(
        real_labels,
        predicted_labels,
        rownames=["Real"],
        colnames=["Previsto"],
    )
    print("\nMatriz de confusão:")
    print(comparison)

    print("\nExemplo, linha 7:")
    print("Valor real:", real_labels[7])
    print("Valor previsto:", predicted_labels[7])

    return {
        "model": model,
        "scores": scores,
        "comparison": comparison,
    }
```

5.3 ÁRVORE DE DECISÃO

Arquivo src/classifiers/tree_decision.py.

```
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report
```

```python
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

from src.clustering.grouping import prepare_data


def tree_classifier(data: pd.DataFrame):
    X, _, y = prepare_data(data)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.33, random_state=125
    )

    # Árvore de decisão. A mesma semente do Naive Bayes deixa a comparação reproduzível.
    # A grade escolhe os hiperparâmetros só no recorte de treino.
    search = GridSearchCV(
        DecisionTreeClassifier(random_state=125),
        {
            "criterion": ["gini", "entropy"],
            "max_depth": [3, 5, 7, 10, None],
            "min_samples_split": [2, 5, 10],
            "min_samples_leaf": [1, 2, 4],
        },
        cv=5,
        scoring="accuracy",
        n_jobs=-1,
    )
    search.fit(X_train, y_train)
    model = search.best_estimator_

    print("\nÁrvore de decisão")
    print("Melhores hiperparâmetros:", search.best_params_)

    # Desempenho pela validação cruzada, com os hiperparâmetros escolhidos
    scores = cross_val_score(
        DecisionTreeClassifier(random_state=125, **search.best_params_),
        X,
        y,
        cv=5,
        scoring="accuracy",
    )
    print("Validação cruzada (5 folds):", scores)
    print("Acurácia média:", scores.mean())
    print("Desvio-padrão:", scores.std())
```

```
48
49      encoder = LabelEncoder()
50      encoder.fit(data["Overall_Impact"])
51
52      predicted = model.predict(X_test)
53      real_labels = encoder.inverse_transform(y_test)
54      predicted_labels = encoder.inverse_transform(predicted)
55
56      print("Acurácia no teste:", accuracy_score(y_test, predicted))
57
58      print("\nRelatório por classe:")
59      print(classification_report(
60          y_test,
61          predicted,
62          target_names=encoder.classes_,
63      ))
64
65      comparison = pd.crosstab(
66          real_labels,
67          predicted_labels,
68          rownames=["Real"],
69          colnames=["Previsto"],
70      )
71      print("\nMatriz de confusão:")
72      print(comparison)
73
74      print("\nExemplo, linha 7:")
75      print("Valor real:", real_labels[7])
76      print("Valor previsto:", predicted_labels[7])
77
78      return {
79          "model": model,
80          "scores": scores,
81          "comparison": comparison,
82      }
```

5.4 GRÁFICOS

Arquivo src/visualization/charts.py.

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  def data_standardization_comparison(data: pd.DataFrame, data_scaled: pd.DataFrame):

5    # Visualização dos dados
6    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
7
8     # Gráfico antes da padronização
9     axes[0].scatter(data["Age"], data["Mental_Health_Index"])
10    axes[0].set_title("Before preprocessing")
11    axes[0].set_xlabel("Age")
12    axes[0].set_ylabel("Mental Health Index")
13
14    # Gráfico após a padronização
15    axes[1].scatter(data_scaled["Age"], data_scaled["Mental_Health_Index"])
16    axes[1].set_title("After standardization")
17    axes[1].set_xlabel("Standardized age")
18    axes[1].set_ylabel("Standardized Mental Health Index")
19
20    # Imagem da comparação antes e após a padronização dos dados
21    plt.savefig("./reports/figures/comparison.png")
22    plt.close(fig)
23
24
25  def classification_charts(data, nb_scores, tree_scores, nb_matrix, tree_matrix):
26    # Contagem das classes de Overall_Impact
27    order = ["Beneficial", "Neutral", "Negative"]
28    counts = data["Overall_Impact"].value_counts().reindex(order)
29
30    fig, ax = plt.subplots(figsize=(6, 4))
31    bars = ax.bar(order, counts.values, color="#4C78A8")
32    ax.bar_label(bars, padding=3)
33    ax.set_ylim(0, counts.max() * 1.12)
34    ax.set_title("Classes de Overall Impact")
35    ax.set_xlabel("Classe")
36    ax.set_ylabel("Registros")
37    fig.tight_layout()
38    fig.savefig("./reports/figures/class_counts.png", dpi=150)
39    plt.close(fig)
40
41    # Acurácia das cinco partes da validação cruzada
42    parts = ["1", "2", "3", "4", "5"]
43    positions = range(len(parts))
44    width = 0.35
45
46    fig, ax = plt.subplots(figsize=(8, 4))
47    nb_bars = ax.bar(
48      [position - width / 2 for position in positions],
49      nb_scores,
50      width,
51      label="Naive Bayes",
52      color="#4C78A8",
53    )
```

```
54  tree_bars = ax.bar(
55    [position + width / 2 for position in positions],
56    tree_scores,
57    width,
58    label="Árvore de decisão",
59    color="#F58518",
60  )
61  ax.bar_label(nb_bars, fmt="%.3f", padding=3, fontsize=8)
62  ax.bar_label(tree_bars, fmt="%.3f", padding=3, fontsize=8)
63  ax.set_xticks(list(positions))
64  ax.set_xticklabels(parts)
65  ax.set_ylim(0.85, 1.02)
66  ax.set_title("Validação cruzada")
67  ax.set_xlabel("Parte")
68  ax.set_ylabel("Acurácia")
69  ax.legend()
70  fig.tight_layout()
71  fig.savefig("./reports/figures/cross_validation.png", dpi=150)
72  plt.close(fig)
73
74  # Matrizes de confusão do recorte de teste
75  labels = ["Beneficial", "Negative", "Neutral"]
76  matrices = (
77    (nb_matrix.reindex(index=labels, columns=labels), "Naive Bayes"),
78    (tree_matrix.reindex(index=labels, columns=labels), "Árvore de decisão"),
79  )
80
81  fig, axes = plt.subplots(1, 2, figsize=(10, 4))
82  for ax, (matrix, title) in zip(axes, matrices):
83    values = matrix.to_numpy()
84    ax.imshow(values, cmap="Blues")
85    ax.set_xticks(range(len(labels)))
86    ax.set_yticks(range(len(labels)))
87    ax.set_xticklabels(labels)
88    ax.set_yticklabels(labels)
89    ax.set_xlabel("Previsto")
90    ax.set_ylabel("Real")
91    ax.set_title(title)
92    darkest = values.max() / 2
93    for row in range(values.shape[0]):
94      for column in range(values.shape[1]):
95        color = "white" if values[row, column] > darkest else "black"
96        ax.text(
97          column,
98          row,
99          int(values[row, column]),
100         ha="center",
```

```
            va="center",
            color=color,
        )
    fig.tight_layout()
    fig.savefig("./reports/figures/confusion_matrices.png", dpi=150)
    plt.close(fig)
```

REFERÊNCIAS

[1] YADAV, Harish. **Impact of Social Media on Life**. 2026. Conjunto de dados. Disponível em: https://www.kaggle.com/datasets/harishyadav0506/impact-of-social-media-on-life. Acesso em: 9 nov. 2026.